



5 DE AGOSTO DE 2026

ARQUITECTURA DE SOLUCION

SISTEMA DE BANCA POR INTERNET

JOSÉ LUIS RODRÍGUEZ



Contenido

Resumen Ejecutivo.....	2
Objetivo General	2
Solución Planteada	2
1. Arquitectura de Negocio.....	2
1.1. Actores	4
1.2. Requerimientos funcionales	4
1.3. Requerimientos no funcionales	5
1.4. Restricciones	5
1.5. Riesgos	5
1.6. Motivación Arquitectónica.....	6
1.7. Mapa de Capacidades.....	7
2. Arquitectura de aplicaciones	8
2.1. Contexto de la solución.....	8
2.2. Contenedores.....	9
2.3. Componentes.....	12
3. Arquitectura de Datos	19
3.1. Base de datos de auditoría	20
4. Arquitectura de Integración	20
4.1. Integración con el Core Bancario	20
4.2. Integración con el BCE	21
4.3. Integración con notificaciones	21
4.4. Integración Registro Civil	21
4.5. Integración con Logikard	22
5. Arquitectura Tecnológica.....	22
5.1. Alta disponibilidad y redundancia	23
5.2. Balanceo de carga y auto-healing.....	23

Resumen Ejecutivo

La solución propone una plataforma de banca por internet utilizando Arquitectura Empresarial para apoyar en la construcción de la solución, ArchiMate para la crear el modelo y la trazabilidad de todo el diseño, C4 para el diseño técnico. La solución incorpora Angular, Flutter, API Gateway, microservicios, OAuth 2.0/OpenID Connect, Logikard para onboarding biométrico, Stratio para consultas de alto volumen y Microsoft Azure como plataforma de despliegue. El diseño prioriza disponibilidad, seguridad, observabilidad y cumplimiento de LOPDP y PCI DSS.

Objetivo General

Diseñar la Arquitectura de Solución de la plataforma de banca por internet, articulando la arquitectura de negocio, de aplicaciones, de integración, de tecnología y de seguridad bajo un enfoque de Arquitectura Empresarial.

Arquitectura de Solución Planteada

Comprensión de la problemática.

El ejercicio plantea la construcción de un sistema de banca por internet en el que los clientes puedan consultar el histórico de sus movimientos y ejecutar transferencias entre cuentas propias e interbancarias. Antes de proponer cualquier componente tecnológico es necesario precisar quiénes intervienen en esos procesos, qué reglas de negocio deben satisfacerse y bajo qué restricciones debe operar la solución.

1. Arquitectura de Negocio

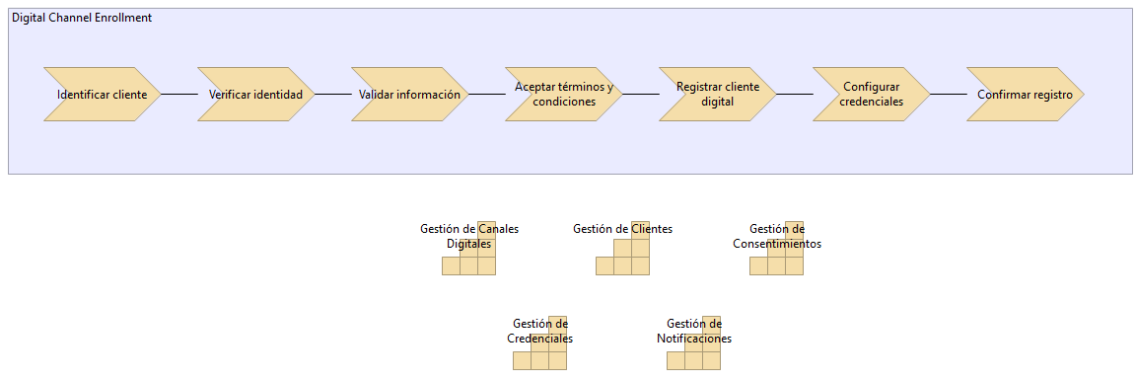
La Arquitectura de Negocio es la representación de cómo una organización crea valor para sus clientes, definiendo sus flujos, capacidades, actores y reglas de negocio, independientemente de la tecnología que se utilice.

Para este ejercicio se ha utilizado el modelo de Value Streams, el cual permite organizar la experiencia del cliente en tres flujos independientes, cada uno con un propósito de negocio y con su conjunto de capacidades. Esta separación apoyará a desarrollar la solución y poder agrupar capacidades que permitirán relacionar con temas normativos.

Flujo de valor: Digital Channel Enrollment

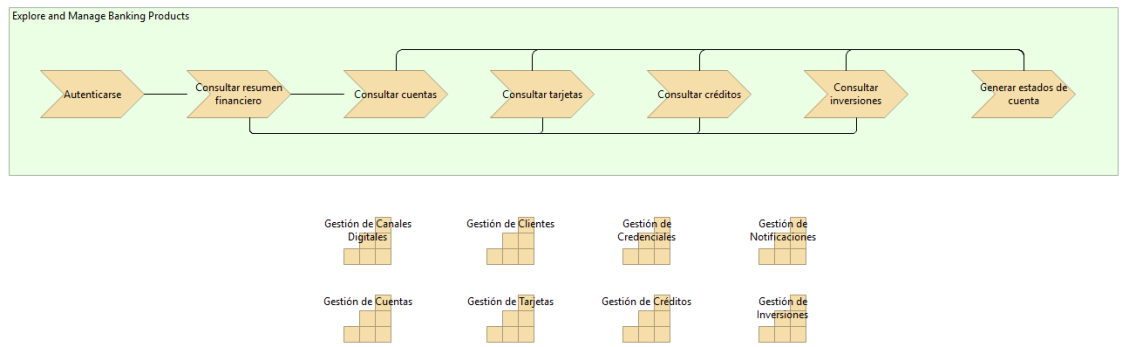
Este flujo de valor tiene como propósito convertir a una persona en cliente digital habilitado para operar en la plataforma. Incluye la identificación del cliente, la

verificación de su identidad mediante reconocimiento facial, la validación de su información contra los sistemas de registro.



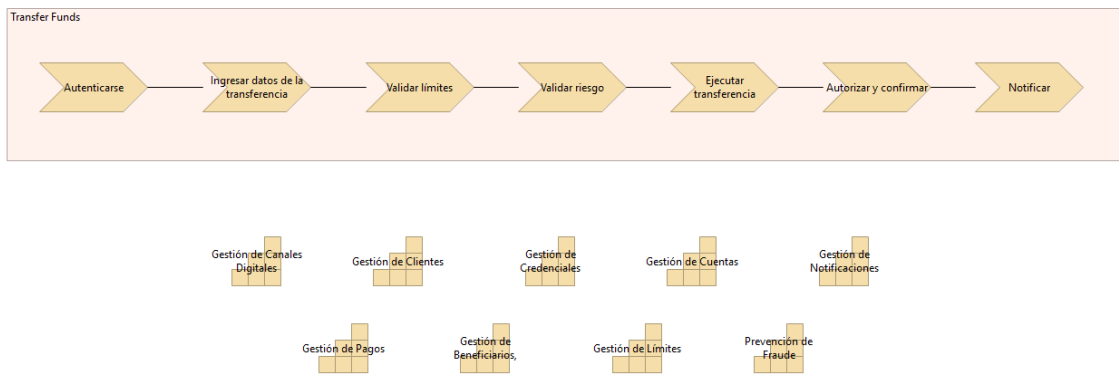
Flujo de valor: Explore Manage Banking Products

Este flujo de valor resuelve la necesidad más frecuente del cliente ya enrolado: conocer el estado de sus productos financieros. Incluye consulta de productos financieros, que abarca cuentas, tarjetas, créditos e inversiones, y que debe resolverse con baja latencia al tratarse de la interacción más frecuente del cliente con la plataforma.



Flujo de valor: Transfer Funds

Este flujo de valor materializa el segundo objetivo funcional del ejercicio: permitir al cliente mover fondos entre cuentas propias e interbancarias. Incluye ejecución de transferencias, que exige validar límites y riesgo antes de autorizar el movimiento, ejecutar la transferencias locales e interbancarias y notificar a la cliente una vez confirmada la operación.



1.1. Actores

Como parte del entendimiento del ejercicio se ha identificado al principal actor Cliente, el cual tiene dos roles con necesidades diferenciadas: el nuevo cliente, que debe completar un proceso de enrolamiento digital antes de poder operar, y el cliente existente, que accede directamente a los servicios de consulta y transferencia.

A estos roles se suman los actores externos con los que la solución debe integrarse:

- Banco Central del Ecuador, que procesa las transferencias interbancarias en lote a través del Sistema de Pagos Interbancarios;
- BANRED, que procesa transferencias interbancarias en tiempo real;
- Logikard, proveedor ecuatoriano de soluciones de registro y validación de datos biométricos de clientes.

Registro Civil del Ecuador, encargado de proveer atender requerimientos para la validación de la identidad de la cédula de ciudadanía ecuatoriana.

- Proveedores de servicios de notificación por SMS y por correo electrónico, requeridos por la norma para informar al cliente sobre los movimientos realizados en su cuenta, pero para este ejercicio no hemos colocado un proveedor específico.

1.2. Requerimientos funcionales

La solución debe permitir el acceso de los clientes desde dos canales, una aplicación web y una aplicación móvil. Debe soportar el onboarding de nuevos clientes con reconocimiento facial y ofrecer, una vez completado ese proceso, al menos un segundo factor de acceso como usuario y clave, huella dactilar u otro mecanismo equivalente. Debe exponer la consulta de datos básicos del cliente, la consulta de movimientos y la ejecución de transferencias, y debe registrar

evidencias de auditoría todas las acciones relevantes que el cliente ejecute sobre la plataforma.

1.3. Requerimientos no funcionales

El ejercicio exige de manera explícita alta disponibilidad, tolerancia a fallos, recuperación ante desastres, seguridad y monitoreo continuo. A ello se suma la exigencia implícita de baja latencia en los servicios de consulta, dado que la entidad cuenta con presupuesto para infraestructura en la nube orientada a ese fin, y la necesidad de que la arquitectura sea desacoplada, con componentes reutilizables.

1.4. Restricciones

La información del cliente proviene de dos sistemas distintos -el Core Bancario y un sistema independiente de detalle de cliente- que deben tratarse como sistemas de registro externos a la solución, sin que el ejercicio contemple su reemplazo.

El servicio de autenticación OAuth 2.0 corresponde a un producto ya adquirido por la compañía que solo requiere configuración, lo que descarta el desarrollo de un servidor de autorización propio.

Las transferencias interbancarias deben canalizarse necesariamente a través de BANRED para el modelo en tiempo real y del Banco Central del Ecuador para el modelo en lote, por lo que la arquitectura de integración debe adaptarse a los protocolos que cada uno de estos actores requieren.

1.5. Riesgos

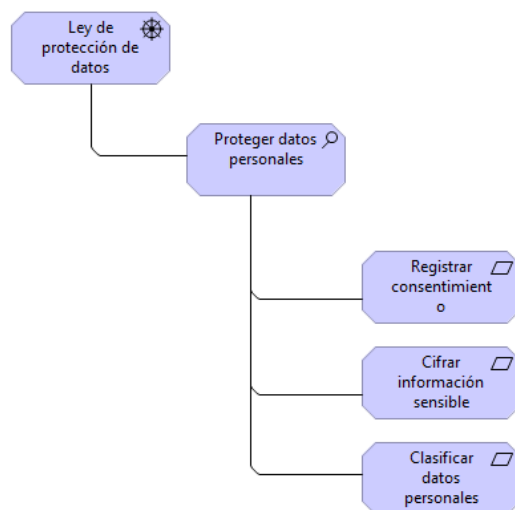
- Dependencia de la disponibilidad del Core Bancario para resolver las consultas de mayor volumen, lo que puede degradar la experiencia del canal digital si no se introduce una capa intermedia de acceso a datos.
- Exposición a la indisponibilidad de actores externos -BANRED, BCE, Logikard, Registro Civil, proveedores de notificación.
- Tratamiento de datos personales y de datos de medios de pago no incorpore desde el diseño los controles exigidos por la LOPDP y por PCI DSS, lo que expondría a la entidad a sanciones regulatorias y a pérdida de confianza de sus clientes

1.6. Motivación Arquitectónica

Es la pieza del modelo que permite documentar que la regulación normativa es incorporada desde la Arquitectura de Negocio, para este ejercicio se ha identificado dos drivers regulatorios explícitos: la Ley de protección de datos personales y el cumplimiento del estándar PCI DSS. Cada uno de estos drivers se descompone en un assessment y en un conjunto de requerimientos técnicos concretos: el driver expresa la fuerza externa que obliga a actuar, el assessment expresa el análisis de esa fuerza sobre la solución, y el requirement expresa la obligación técnica derivada.

Driver: Ley de protección de datos personales

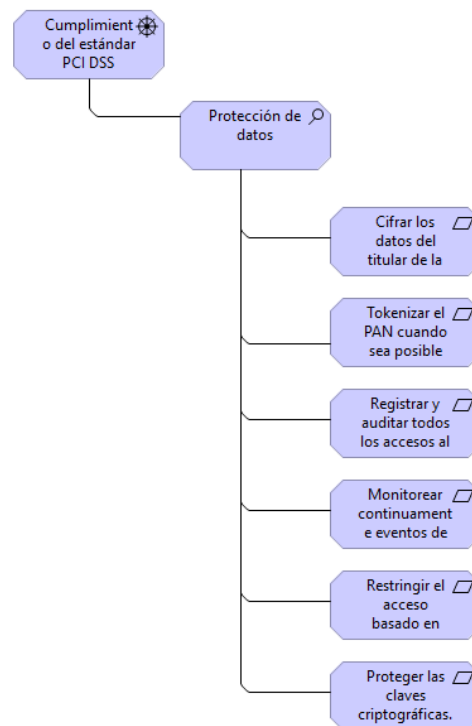
Este driver se traduce en el assessment de proteger los datos personales de los clientes durante todo su ciclo de vida en la plataforma, y de ese assessment se desprenden tres requerimientos: registrar el consentimiento del cliente, cifrar la información sensible y clasificar los datos personales según su nivel de sensibilidad. El modelo vincula este driver directamente con la capacidad de Gestión de Clientes y con la capacidad de Gestión de Beneficiarios, considerando tanto el perfil del cliente como los datos de las personas beneficiarias de una transferencia constituyen datos personales sujetos a la norma, y por tanto ambos dominios de capacidad deben construirse con controles de cifrado, clasificación y consentimiento.



Driver: Cumplimiento del estándar PCI DSS

Este segundo driver se traduce en el assessment de protección de datos de medios de pago, del cual se desprenden seis requerimientos: cifrar los datos del titular de la tarjeta, tokenizar el número de cuenta primario (PAN) cuando sea posible, registrar y auditar todos los accesos al entorno que tengan el número de tarjeta,

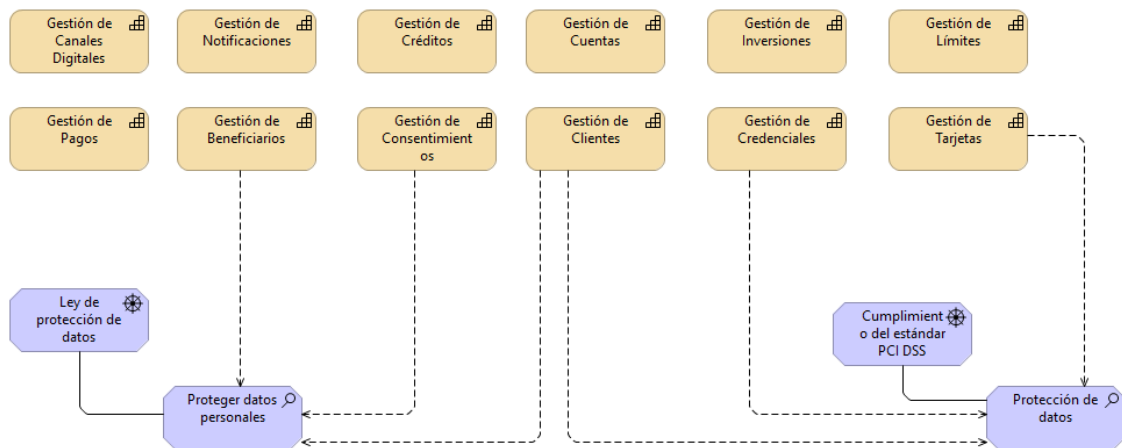
monitorear de forma continua los eventos de seguridad, restringir el acceso con base en roles y proteger las claves criptográficas utilizadas. El modelo vincula este driver con las capacidades de Gestión de Clientes, Gestión de Credenciales y Gestión de Tarjetas, lo que anticipa con precisión dónde debe concentrarse el esfuerzo de cumplimiento: el servicio de tarjetas, por manejar directamente datos de titular; el servicio de credenciales, por custodiar los factores de autenticación que protegen el acceso a esos datos; y el dominio de clientes, por ser el punto donde ambos tipos de información convergen.



1.7. Mapa de Capacidades

El mapa de capacidades construido para este ejercicio es la traducción de los tres flujos de valor descritos a un lenguaje independiente de la tecnología que los resuelve, cada capacidad debe tener independencia de si hoy la resuelve un módulo del Core Bancario, un servicio nuevo o un producto de terceros.

Durante la construcción del mapa se ha identificado que capacidad de Gestión de Clientes aparece asociada tanto al flujo de enrolamiento como al de consulta y al de transferencia, lo que la convierte en una capacidad transversal que sostiene la identidad del cliente en cualquier interacción, y por tanto no debe implementarse de forma redundante en cada dominio de aplicación sino centralizarse en un servicio compartido.

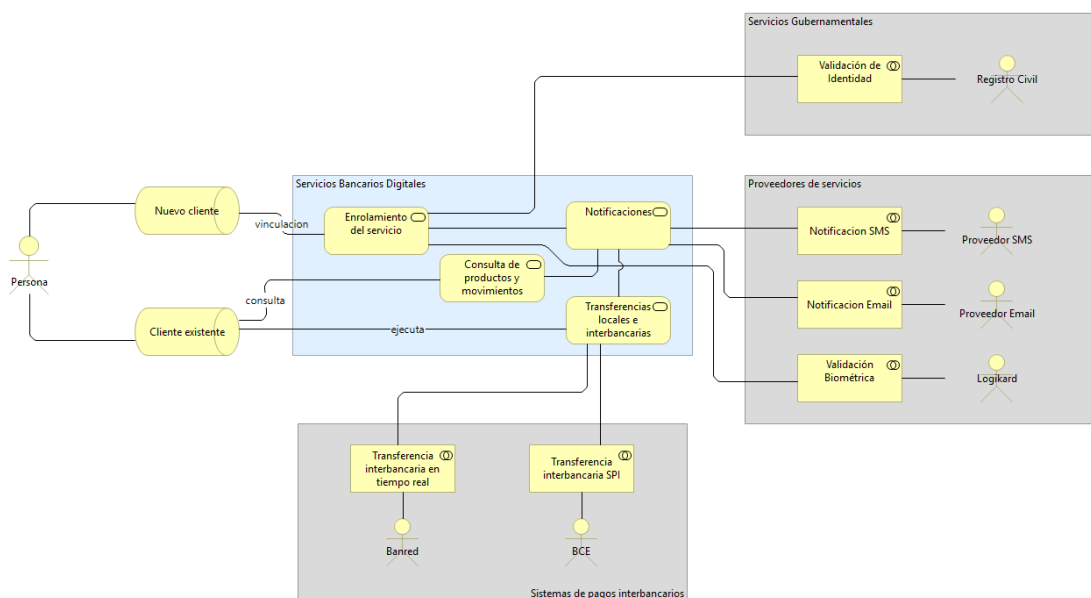


2. Arquitectura de aplicaciones

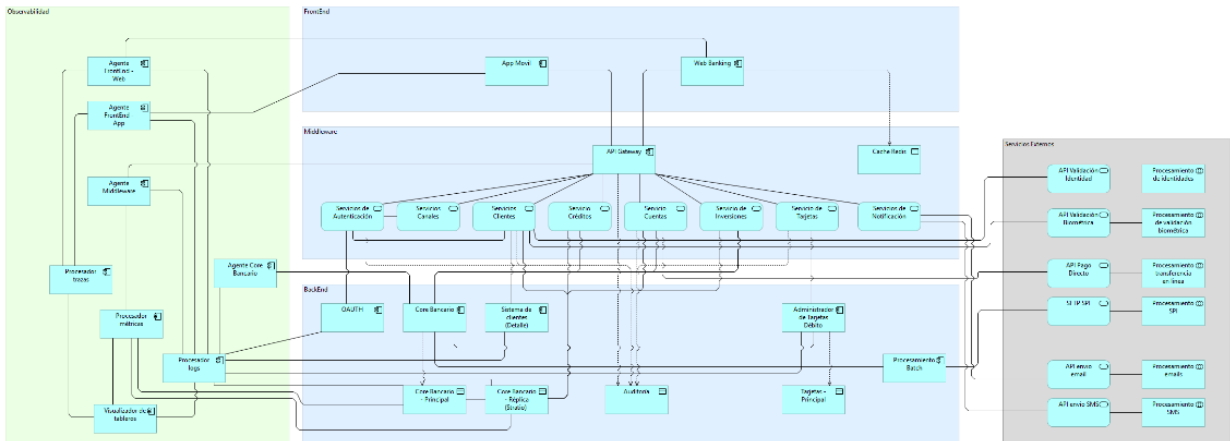
La Arquitectura de Aplicaciones describe el conjunto de aplicaciones que soportan los flujos de valor y capacidades del negocio, definiendo sus responsabilidades, las interfaces entre ellas y la forma en que colaboran para ofrecer los servicios requeridos por la organización.

2.1. Contexto de la solución

El objetivo de este diagrama es definir el alcance del sistema, los actores que interactúan con él y las integraciones externas que soportan los principales procesos de negocio.



2.2. Contenedores



2.2.1. Capa FrontEnd

Esta capa agrupa la App Móvil y la Web Banking, los dos canales digitales exigidos por el ejercicio. Ambos componentes son consumidores de la capa de Middleware y no contienen lógica de negocio: su responsabilidad se limita a la presentación, a la orquestación de la experiencia de usuario y a la comunicación segura con el API Gateway. Esta separación es la que permite que el equipo móvil evolucione su aplicación en un framework multiplataforma sin coordinar cada cambio con el equipo del canal web, siempre que ambos respeten el contrato expuesto por la capa de Middleware.

Selección de tecnología para Web Banking: Angular frente a React

Angular y React resuelven el mismo requerimiento: construir una interfaz de una sola página; pero parten de filosofías distintas: Angular es un framework completo y opinado que integra enrutamiento, formularios reactivos, cliente HTTP, inyección de dependencias y utilidades de pruebas dentro de una misma distribución oficial, mientras que React es una librería de interfaz que delega en el equipo la elección de cada una de esas piezas.

Criterio	Angular	React
Naturaleza	Framework completo y <i>opinionated</i>	Librería de interfaz, requiere ensamblar el resto del stack

Criterio	Angular	React
TypeScript	Nativo desde su diseño	Soporte opcional
Gobierno en equipos grandes	Convenciones estrictas de módulos, servicios e inyección de dependencias	Mayor libertad; el equipo debe definir sus propias convenciones
Protección frente a XSS en la interfaz	Sanitización de HTML integrada por defecto contra XSS	Escapado seguro del contenido renderizado por defecto; el HTML dinámico debe ser confiable y sanitizado antes de usarlo
Capacidades integradas	Incluye alternativas oficiales para routing, formularios, HTTP, compilación y pruebas.	Núcleo más pequeño y componible; las capacidades adicionales se incorporan mediante frameworks o librerías.
Curva de aprendizaje	Mayor inversión inicial por la cantidad de conceptos y convenciones; comportamiento más homogéneo al escalar.	Inicio más gradual; la complejidad de decisiones arquitectónicas aumenta conforme crece la aplicación

Para el Web Banking se recomienda **Angular** porque proporciona una estructura más *opinionated*, capacidades integradas y convenciones homogéneas que facilitan el gobierno técnico, la trazabilidad y la consistencia entre equipos grandes. React también es una alternativa válida y segura, pero requiere definir con mayor detalle el framework, las librerías y los estándares corporativos que conformarán la arquitectura completa.

Selección de tecnología para la App Móvil: Flutter frente a React Native

Flutter y React Native resuelven el requerimiento de construir una aplicación nativa para iOS y Android desde una sola base de código, pero con estrategias de renderizado distintas: Flutter compila a código nativo y dibuja su interfaz con un motor de renderizado propio, independiente de los componentes nativos del sistema operativo, mientras que React Native traduce sus componentes hacia elementos nativos de cada plataforma.

Criterio	Flutter	React Native
Lenguaje	Dart	JavaScript / TypeScript

Criterio	Flutter	React Native
Renderizado	Motor propio (Skia/Impeller), consistente en ambas plataformas	Se apoya en componentes nativos de cada plataforma
Consistencia visual iOS/Android	Alta, un mismo motor en ambos sistemas	Puede variar sutilmente entre plataformas
Reutilización de conocimiento del equipo web	Baja si el equipo web usa Angular/TypeScript; requiere aprender Dart	Alta, reutiliza JavaScript/TypeScript del equipo web
Ecosistema de SDKs biométricos y de pagos	Creciente, con soporte activo de Google	Más maduro históricamente en integraciones fintech

Se recomienda Flutter como la tecnología a adoptar. La razón principal es que el proceso de onboarding con reconocimiento facial y el flujo de transferencias son las dos interacciones más sensibles de la aplicación, y ambas se benefician de una experiencia visual idéntica entre iOS y Android; delegar el renderizado a los componentes nativos de cada plataforma, como hace React Native, introduce variaciones sutiles de comportamiento que son más difíciles de probar exhaustivamente en un flujo regulado. El costo de esta decisión es que el equipo debe incorporar Dart como un lenguaje adicional al TypeScript que ya usa en Angular; se considera un costo asumible frente a la ganancia en previsibilidad de la experiencia de captura biométrica.

2.2.2. Capa Middleware

Esta es la capa que concentra el API Gateway junto con los servicios de aplicación. El API Gateway es el único punto de entrada desde los canales hacia estos servicios, lo que permite centralizar en un solo componente las políticas de autenticación, de límite de tasa y de enrutamiento, en lugar de duplicarlas en cada servicio individual. Cada servicio de la capa Middleware corresponde de manera directa a una capacidad de negocio del mapa de capacidades del capítulo anterior, lo que confirma que la descomposición de servicios no responde a un criterio técnico arbitrario sino a la estructura de capacidades ya validada con el negocio.

2.2.3. Capa BackEnd

Esta capa agrupa los sistemas: Core Bancario, Sistema de clientes en su versión de Detalle, componente de Procesamiento Batch y el componente OAUTH, que representa el producto de autenticación ya adquirido por la compañía y que los servicios de Middleware consumen para validar tokens, sin necesidad de implementar la lógica del protocolo. La ubicación de OAUTH en la capa BackEnd, y

no en Middleware, es una decisión de interpretación relevante: aunque de cara a los canales el resultado de la autenticación se percibe como un servicio de Middleware, el componente que efectivamente emite y valida los tokens es un sistema de registro de identidad que debe tratarse con el mismo rigor operativo que el Core Bancario.

2.2.4. Observabilidad

La Capa de Observabilidad proporciona las capacidades necesarias para supervisar en tiempo real el estado operativo de la plataforma de banca digital, este agrupa los agentes de instrumentación por capa junto con los procesadores de trazas, métricas y logs y el Visualizador de tableros.

2.2.5. Servicios Externos

La Capa de Servicios Externos agrupa los sistemas y proveedores ajenos a la solución de banca digital que participan en la ejecución de procesos del negocio mediante mecanismos de integración.

2.3. Componentes

Los diagramas de componentes se encuentran organizados por cada Flujo de Valor identificado en la Arquitectura de Negocio, permitiendo mantener la trazabilidad entre las necesidades del negocio y los componentes de software que las implementan. Cada diagrama representa los servicios, componentes internos, repositorios de datos e integraciones externas necesarios para soportar cada flujo.

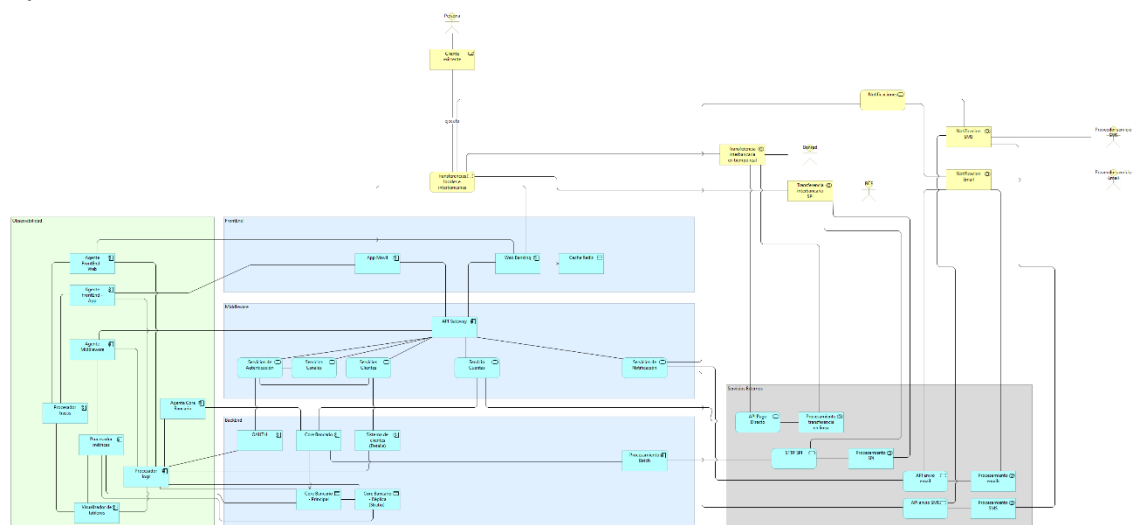
2.3.1. Componentes para Transferencias

El diagrama representa los componentes que implementan el flujo de valor "Transferencias Locales e Interbancarias", describiendo la colaboración entre los canales digitales, los servicios de negocio, los sistemas bancarios y los proveedores externos para garantizar una ejecución segura, resiliente y trazable de las transacciones financieras. El cliente inicia la operación desde la aplicación móvil o el Web Banking, que envían la solicitud al API Gateway. Este componente centraliza el acceso a la plataforma, aplica las políticas de seguridad y direcciona la petición hacia los servicios especializados de autenticación, clientes, cuentas y notificaciones, responsables de validar la identidad del usuario, verificar la información de las cuentas y coordinar la ejecución de la transferencia.

Una vez autorizada la operación, el Servicio de Cuentas determina el tipo de transferencia a ejecutar. Las transferencias entre cuentas del mismo banco son procesadas directamente por el Core Bancario, mientras que las transferencias interbancarias utilizan dos mecanismos de integración. Para las operaciones en tiempo real, el sistema consume la API de Pago Directo, la cual establece la comunicación con BANRED para el procesamiento inmediato de la transacción. Para las transferencias diferidas o por lotes, el sistema genera el archivo conforme al estándar SPI, el cual es enviado al Banco Central del Ecuador (BCE) mediante el servicio de procesamiento batch. Esta separación permite soportar ambos esquemas de compensación sin afectar la lógica de negocio de la solución.

Una vez confirmada la transacción, el Servicio de Notificaciones coordina el envío de mensajes al cliente utilizando los servicios externos de correo electrónico y SMS. La solución desacopla la lógica de negocio de los mecanismos de comunicación mediante APIs especializadas para el envío de notificaciones, permitiendo incorporar nuevos canales de comunicación sin afectar el resto de la arquitectura.

De forma transversal, la Capa de Observabilidad instrumenta todos los componentes del flujo mediante agentes distribuidos que recopilan métricas, logs y trazas desde los canales digitales, el middleware, el Core Bancario y las integraciones externas. Esta información es procesada y centralizada para proporcionar monitoreo en tiempo real, trazabilidad completa de las transacciones, generación de alertas y análisis de desempeño, permitiendo reducir los tiempos de detección y resolución de incidentes en un proceso crítico para la operación bancaria..

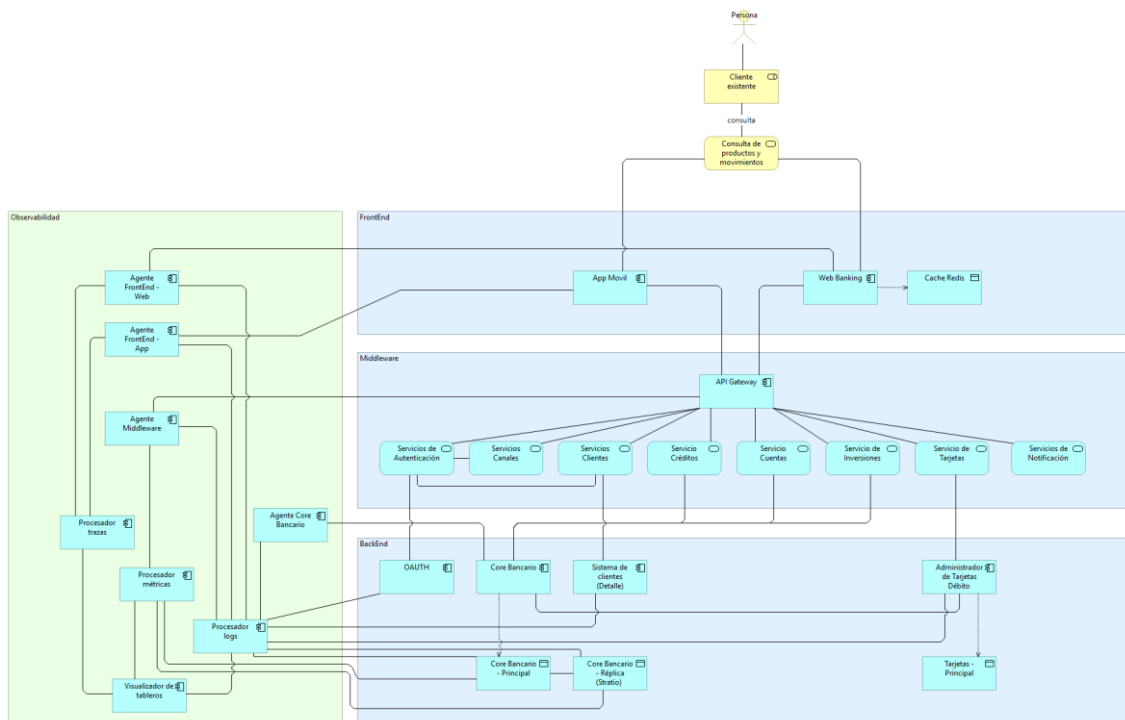


2.3.2. Componentes para Consultas

El diagrama representa los componentes que soportan el flujo de valor “Consulta de productos y movimientos” para el cliente existente. El acceso se realiza desde la aplicación móvil o el Web Banking, que canalizan las solicitudes a través del API Gateway. Este componente centraliza el ingreso a la plataforma, aplica controles de seguridad y enruta cada petición hacia el servicio correspondiente, como Clientes, Cuentas, Créditos, Inversiones o Tarjetas. El servicio de Autenticación valida la identidad y los permisos del usuario mediante el proveedor OAuth, mientras que el Servicio de Canales coordina la interacción con los canales digitales.

Los servicios de negocio consultan la información en los sistemas fuente según el tipo de producto. El Core Bancario atiende principalmente las consultas de cuentas, movimientos, créditos e inversiones, mientras que el Sistema de Clientes proporciona información complementaria y el Administrador de Tarjetas de Débito gestiona los datos específicos de tarjetas. Para reducir la carga sobre el Core Bancario y mejorar los tiempos de respuesta, las consultas de alta frecuencia pueden apoyarse en una réplica de lectura basada en Stratio y en mecanismos de caché, como Redis, evitando acceder directamente a la base principal cuando la información no requiere consistencia transaccional inmediata.

La observabilidad se implementa como una capacidad transversal. Los agentes del frontend, middleware, Core Bancario y aplicación móvil recopilan métricas, trazas y logs, que son procesados y presentados en tableros centralizados. Esto permite monitorear la disponibilidad, la latencia, los errores y el recorrido completo de cada consulta desde el canal digital hasta el sistema de origen, facilitando la detección temprana de incidentes y el análisis de rendimiento.



2.3.3. Componentes de enrolamiento

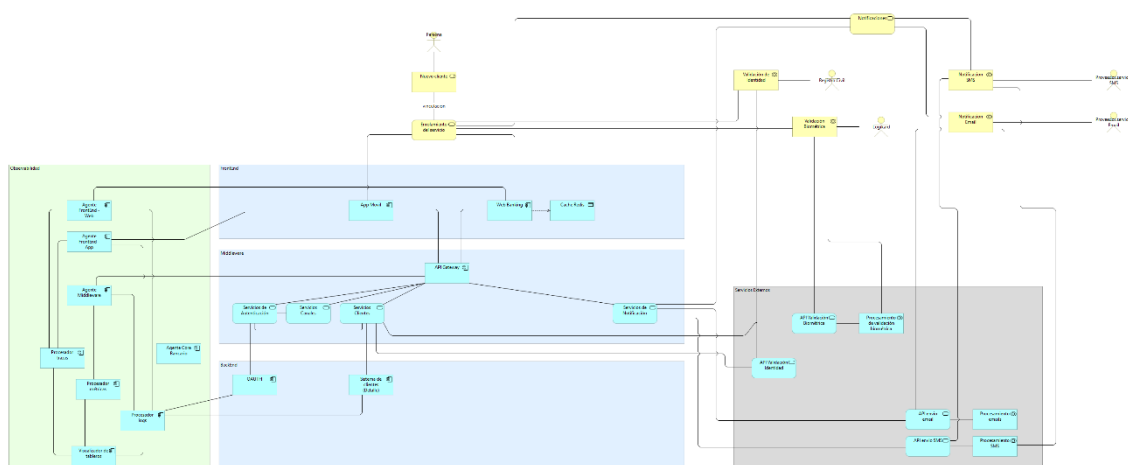
El diagrama representa los componentes que implementan el flujo de valor de “Enrolamiento del servicio” para un nuevo cliente. El proceso inicia en la aplicación móvil, desde donde el usuario registra su información y envía las evidencias requeridas para la vinculación digital. La solicitud es canalizada a través del API Gateway, que centraliza el acceso, aplica controles de seguridad y dirige la operación hacia los servicios de autenticación, canales, clientes y notificaciones. Estos servicios coordinan la creación del usuario digital, la validación de sus datos y la habilitación de sus mecanismos de acceso.

Durante el enrolamiento, el Servicio de Clientes consulta el Sistema Complementario de Información del Cliente para verificar o completar los datos disponibles. La validación de identidad se realiza mediante una API especializada que integra la solución con el Registro Civil, mientras que la comprobación biométrica se ejecuta a través de una API de validación facial conectada con el proveedor Logikard. Esta separación desacopla la lógica de onboarding de los proveedores externos y permite sustituir o incorporar nuevos servicios de identidad y biometría sin afectar los canales digitales ni los servicios centrales.

Una vez superadas las validaciones de identidad y biometría, el Servicio de Autenticación registra o habilita al nuevo usuario en la plataforma OAuth,

permitiendo posteriormente el acceso mediante usuario y contraseña, biometría del dispositivo u otros factores de autenticación autorizados. Al finalizar el proceso, el Servicio de Notificaciones comunica al cliente el resultado del enrolamiento mediante correo electrónico y SMS, utilizando APIs independientes para cada proveedor.

La Capa de Observabilidad participa de forma transversal en todo el flujo mediante agentes instalados en la aplicación móvil, el middleware y los sistemas bancarios. Estos componentes recopilan métricas, logs y trazas distribuidas, que son procesadas y visualizadas en tableros centralizados. De esta manera, la solución mantiene trazabilidad de extremo a extremo, permite identificar fallos en las validaciones externas y facilita el monitoreo del tiempo de respuesta, la disponibilidad y la tasa de éxito del proceso de enrolamiento.



2.3.4. Autenticación

Aunque el banco cuenta con un producto OAuth 2.0/OpenID Connect ya adquirido y que solo requiere configuración, la selección del flujo de autorización sigue siendo una decisión de arquitectura que condiciona la seguridad de ambos canales. Tanto la App Móvil como el Web Banking son clientes públicos: no pueden custodiar un client secret de forma segura, ya que su código se ejecuta en el dispositivo o navegador del usuario y por tanto es potencialmente inspeccionable.

Por esta razón se recomienda que ambos canales utilicen el flujo Authorization Code con PKCE (Proof Key for Code Exchange, RFC 7636), que es hoy el estándar de la industria para clientes públicos y el que recomiendan tanto OAuth 2.1 como las guías de mejores prácticas para aplicaciones nativas (RFC 8252). Se descartan el flujo Implicit, porque expone el token de acceso directamente en el fragmento de la

URL sin posibilidad de renovarlo de forma segura y está deprecado en la práctica de la industria; el flujo Resource Owner Password Credentials, porque obligaría a que la aplicación capture y transmita la contraseña del cliente, comprometiendo la separación entre canal digital y proveedor de identidad; y el flujo Client Credentials, que corresponde a comunicación servicio a servicio y no a la autenticación de un usuario final.

Con Authorization Code y PKCE, el canal redirige al usuario al endpoint de autorización de OAUTH junto con un code_challenge derivado de un secreto generado localmente (code_verifier); una vez el usuario se autentica, OAUTH retorna un código de un solo uso a la URI de redirección registrada -un custom scheme o App Link en la App Móvil, y una ruta propia del dominio en el Web Banking-, y el canal intercambia ese código junto con el code_verifier original por el access token y el refresh token. Este intercambio impide que un tercero que intercepte el código de autorización pueda canjearlo por tokens sin conocer el code_verifier.

Para el Web Banking se recomienda que el intercambio de tokens y su almacenamiento no ocurra directamente en el navegador, sino a través de un patrón Backend-for-Frontend que retenga los tokens del lado del servidor y exponga al navegador únicamente una cookie de sesión httpOnly y Secure; esto reduce la superficie de robo de tokens ante un eventual XSS, complementando la sanitización que Angular aplica por defecto. Para la App Móvil, los tokens deben almacenarse en el almacenamiento seguro provisto por el sistema operativo -Keychain en iOS y Keystore en Android-, y los access tokens deben tener una vida corta, apoyándose en la renovación mediante refresh token con rotación, de forma que un token filtrado pierda validez rápidamente.

El segundo factor de acceso posterior al onboarding -usuario y clave, huella dactilar u otro mecanismo equivalente, según lo exige el ejercicio- se recomienda resolverlo como autenticación local del dispositivo orquestada como step-up hacia OAUTH, y no como un factor gestionado íntegramente por el canal: en la App Móvil, mediante los proveedores biométricos nativos del sistema operativo -Face ID/Touch ID en iOS y BiometricPrompt en Android-, expuestos en Flutter a través de plugins como local_auth, donde una validación biométrica exitosa habilita el uso del refresh token almacenado sin solicitar nuevamente la contraseña; en el Web Banking, se recomienda evaluar WebAuthn/FIDO2 como estándar de industria para segundo factor sin contraseña, aprovechando los autenticadores de plataforma del propio dispositivo del usuario, siempre que el producto OAuth ya adquirido por la compañía soporte su integración.

2.3.5. Onboarding y Validación Biométrica

El proceso de onboarding incorpora validación biométrica facial como parte del registro inicial del cliente, garantizando la verificación de su identidad antes de

emitir cualquier credencial de acceso. Este proceso combina la captura del documento de identidad, reconocimiento óptico de caracteres (OCR), detección de vida (liveness) y comparación biométrica entre el rostro del usuario y el documento presentado, reduciendo el riesgo de suplantación de identidad durante la incorporación a la banca digital.

Una vez completado el proceso de registro, el acceso a la plataforma se realiza mediante usuario y contraseña, complementado con un segundo factor basado en las capacidades biométricas nativas del dispositivo, como Face ID, Touch ID o Android BiometricPrompt. Esta aproximación aprovecha los mecanismos de seguridad provistos por los sistemas operativos móviles, evitando desarrollar un motor biométrico propio y reduciendo los riesgos operativos y de mantenimiento.

El componente Verificador Biométrico, descrito en la Arquitectura de Componentes, se implementa mediante la integración con un proveedor especializado de servicios de identidad digital. Esta decisión responde a que capacidades como OCR, validación documental, detección de vida y comparación biométrica requieren algoritmos especializados, certificaciones antifraude y una actualización continua frente a nuevas técnicas de fraude, aspectos que exceden el alcance del proyecto y que son cubiertos de forma más eficiente por soluciones especializadas del mercado.

Como parte del análisis de alternativas se evaluaron las plataformas Logikard Onboarding, FacePhi Identity Platform, Jumio, Veridas y Daon, considerando criterios funcionales y operativos:

Criterio	Logikard	FacePhi	Jumio	Veridas	Daon
Origen y soporte	Ecuador	España	EE.UU.	España	EE.UU./Irlanda
OCR y validación documental	✓	✓	✓	✓	✓
Detección de vida (<i>Liveness</i>)	✓	✓	✓	✓	✓
Comparación rostro-documento	✓	✓	✓	✓	✓
Adaptación al marco KYC ecuatoriano	Alta	Media	Media	Media	Media
Soporte local y SLA	Alta	Media	Media	Media	Media

Dado que todas las soluciones evaluadas cumplen los requerimientos funcionales del proceso de onboarding, la selección se fundamentó principalmente en criterios de operación y soporte. En este contexto, **Logikard** representa la alternativa más adecuada al ofrecer soporte local en español, tiempos de respuesta más cortos, contratación en moneda local y un mayor conocimiento del marco regulatorio ecuatoriano para procesos de identificación digital (KYC). Estos factores reducen los riesgos operativos y facilitan la evolución de la solución sin comprometer las capacidades funcionales requeridas.

No obstante, la arquitectura mantiene un bajo acoplamiento con el proveedor mediante un adaptador de integración, permitiendo reemplazar la plataforma biométrica en el futuro sin afectar el flujo de onboarding ni el resto de los componentes de la solución.

3. Arquitectura de Datos

La Arquitectura de Datos se fundamenta en un principio clave para garantizar el rendimiento y la escalabilidad de la solución: los sistemas de registro (Systems of Record) no deben ser utilizados como fuente directa para consultas masivas de solo lectura. El Core Bancario constituye la fuente oficial de información transaccional, almacenando los datos de clientes, cuentas, productos y movimientos financieros, mientras que el Sistema Complementario de Información del Cliente proporciona información detallada cuando el canal digital requiere datos adicionales del titular.

Ambos sistemas están diseñados para soportar las operaciones transaccionales críticas del banco, donde la consistencia e integridad de los datos son prioritarias. Sin embargo, el acceso directo desde los canales digitales para atender un elevado volumen de consultas concurrentes incrementaría la carga sobre estas plataformas, afectando su desempeño y la capacidad de respuesta de los procesos de negocio. Dado que uno de los objetivos de la solución es ofrecer una experiencia digital ágil y de baja latencia, es necesario desacoplar las operaciones de consulta de los sistemas transaccionales.

Por esta razón, la arquitectura incorpora una plataforma de lectura basada en Stratio, que mantiene una réplica sincronizada de la información del Core Bancario para atender las consultas de productos y movimientos desde los canales digitales. Adicionalmente, se utiliza Redis como mecanismo de caché para almacenar

información de alta frecuencia de consulta, reduciendo la latencia y minimizando el número de accesos a los sistemas de registro.

Con este enfoque se consigue una arquitectura orientada a la separación entre lectura y escritura (CQRS a nivel de infraestructura), donde las operaciones de consulta son atendidas por componentes optimizados para lectura, mientras que las operaciones transaccionales continúan ejecutándose sobre el Core Bancario. Esta estrategia mejora la escalabilidad, protege el rendimiento de los sistemas críticos y proporciona una base sólida para soportar el crecimiento de los canales digitales sin comprometer la estabilidad de la operación bancaria.

3.1. Base de datos de auditoría

Toda acción relevante que el cliente ejecute sobre la plataforma -inicio de sesión, consulta de movimientos, ejecución de una transferencia, cambio de credenciales- se registra en una base de datos de auditoría independiente de los sistemas de registro. Esta separación responde directamente al requerimiento derivado del driver de cumplimiento PCI DSS de registrar y auditar todos los accesos al entorno, y permite que la trazabilidad de la operación sobreviva incluso si el sistema de origen de la transacción cambia en el futuro. La auditoría se diseña como un componente de solo escritura desde la perspectiva de los servicios de negocio, de forma que ningún servicio transaccional dependa de una lectura previa contra la auditoría para completar su operación.

4. Arquitectura de Integración

Toda comunicación entre los canales digitales y los servicios de negocio atraviesa el API Gateway, que actúa como fachada. Esta decisión concentra en un solo componente las responsabilidades de autenticación de la solicitud, enrutamiento hacia el servicio correcto, control de tasa de peticiones y observabilidad del tráfico, evitando que cada servicio de Middleware deba implementar de forma independiente estas capacidades transversales.

4.1. Integración con el Core Bancario

Los servicios de Cuentas, Tarjetas, Créditos e Inversiones se integran con el Core Bancario mediante llamadas síncronas para las operaciones que exigen el dato más reciente -validación de saldo disponible previo a una transferencia- y mediante el mecanismo de sincronización hacia Stratio para las consultas informativas. Esta doble vía de integración es la que permite balancear la exigencia de baja latencia

del ejercicio con la necesidad de no comprometer la consistencia de la información en el momento en que realmente importa: la ejecución de la transferencia.

4.2. Integración con BANRED

La transferencia interbancaria en tiempo real se resuelve mediante dos interfaces API diferenciadas hacia BANRED, tal como se definió en los argumentos de la solución: una API adquirente, que la plataforma de banco invoca para enviar la orden de transferencia hacia BANRED cuando el cliente es quien origina la operación, y una API autorizadora, que BANRED invoca hacia el banco para recibir las transferencias interbancarias en tiempo real destinadas a los clientes de la entidad. Esta separación de roles evita que un mismo contrato de API deba resolver simultáneamente el envío y la recepción de transferencias, lo que simplifica el manejo de errores y de reintentos en cada dirección de forma independiente.

4.3. Integración con el BCE

Las transferencias interbancarias que el BCE procesa en lote se resuelven mediante el componente de Procesamiento Batch, que consolida las órdenes de transferencia ejecutadas durante la ventana correspondiente y genera el archivo bajo el estándar del Sistema de Pagos Interbancarios exigido por el Banco Central. A diferencia de la integración con BANRED, esta vía no ofrece confirmación inmediata al cliente, por lo que la arquitectura debe comunicar con claridad al usuario, desde el canal, que este tipo de transferencia se liquidará en batch y no de forma instantánea.

4.4. Integración con notificaciones

El servicio de Notificaciones se apoya en dos proveedores externos -uno de SMS y uno de correo electrónico- cumpliendo así el requerimiento normativo de contar con un mínimo de dos canales para informar al cliente sobre los movimientos realizados. La integración con estos proveedores debe diseñarse como asíncrona y desacoplada de la transacción de negocio: la confirmación de una transferencia al cliente no debe esperar a que el proveedor de SMS o de correo confirme la entrega del mensaje, sino que debe encolarse para su envío y reintento independiente, de forma que una falla o demora del proveedor de notificación no afecte el tiempo de respuesta percibido por el cliente en la operación principal.

4.5. Integración Registro Civil

La integración con el Registro Civil permite verificar la identidad declarada por el nuevo cliente utilizando la información oficial disponible en la entidad gubernamental. Durante el enrolamiento, el Servicio de Gestión de Clientes remite la solicitud al componente API de Verificación de Identidad, encargado de adaptar

la información al formato requerido por el servicio externo, gestionar la autenticación técnica y procesar la respuesta.

La validación puede considerar datos como el número de cédula, nombres, fecha de nacimiento y otros atributos habilitados por el convenio o interfaz institucional. Cuando corresponda, también podría incluir la verificación mediante código dactilar; sin embargo, su disponibilidad depende de las condiciones y servicios proporcionados por el Registro Civil.

4.6. Integración con Logikard

La integración con Logikard proporciona la capacidad de validación biométrica requerida durante el onboarding digital. Desde la aplicación móvil se capturan las evidencias biométricas solicitadas —por ejemplo, imagen facial, video o prueba de vida— y se transmiten de forma cifrada hacia la plataforma bancaria.

El componente API de Verificación Biométrica desacopla la solución del proveedor y se encarga de:

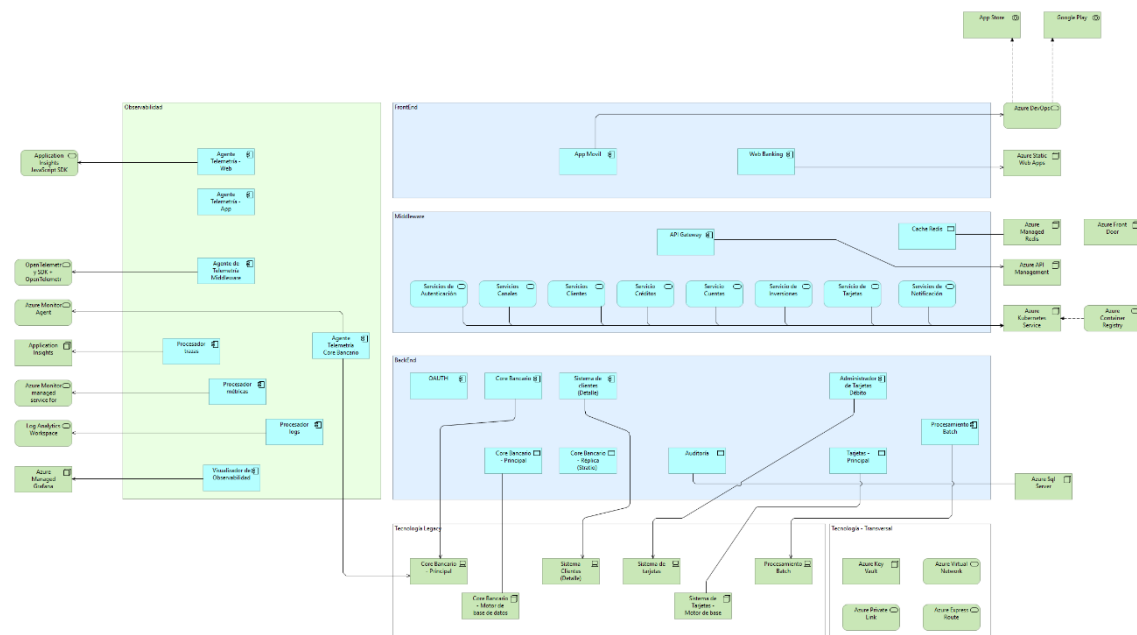
- Validar la integridad y calidad de la evidencia recibida.
- Transformar la solicitud al contrato de integración de Logikard.
- Invocar el servicio de reconocimiento facial y prueba de vida.
- Interpretar el resultado y el nivel de confianza proporcionado.
- Manejar tiempos de espera, reintentos controlados y fallos del proveedor.
- Registrar la trazabilidad técnica sin exponer datos biométricos en los logs.

La respuesta del proveedor es utilizada por el Orquestador de Enrolamiento para aprobar, rechazar o derivar el caso a una validación adicional, de acuerdo con las políticas de riesgo definidas por el banco.

5. Arquitectura Tecnológica

La arquitectura tecnológica utiliza servicios administrados de Azure para reducir la carga operativa y proporcionar capacidades de seguridad, escalabilidad, disponibilidad y recuperación. Los canales digitales se protegen mediante Azure Front Door y WAF; las APIs son gobernadas por Azure API Management; y los servicios de negocio se ejecutan sobre AKS con autoescalado y auto-healing. Azure Service Bus desacopla los procesos asíncronos, Azure Managed Redis optimiza las consultas y Azure Functions y Logic Apps soportan el procesamiento batch y las integraciones especializadas. La observabilidad se implementa mediante Azure Monitor managed service for Prometheus, Azure Managed Grafana, Log Analytics y Application Insights. Los secretos y llaves se administran en Azure Key Vault, mientras que la conexión con los sistemas bancarios existentes se establece por

canales privados. El Core Bancario y las demás plataformas actuales permanecen fuera del alcance de migración y son tratados como sistemas externos integrados.



5.1. Alta disponibilidad y redundancia

Siguiendo el argumento de la solución de aprovechar las oportunidades de la nube mediante redundancia regional y zonal, se recomienda desplegar los servicios de Middleware y el API Gateway en al menos dos zonas de disponibilidad dentro de una misma región, de forma que la pérdida de un centro de datos no interrumpa el servicio, y replicar la infraestructura crítica -API Gateway, servicios de autenticación y caché- en una segunda región como estrategia de recuperación ante desastres. Esta redundancia regional no aplica de la misma manera al Core Bancario, cuya topología de alta disponibilidad depende de la arquitectura propia de ese sistema y excede el alcance de este ejercicio, pero sí debe considerarse en el diseño de los conectores hacia el Core para que toleren una conmutación de nodo sin perder transacciones en curso.

5.2. Balanceo de carga y auto-healing

La solución implementa mecanismos de balanceo de carga y auto-healing para garantizar la continuidad del servicio, distribuir eficientemente las solicitudes de los usuarios y minimizar el impacto de fallos en la plataforma.

El acceso a los canales digitales se realiza a través de Azure Front Door, que actúa como punto de entrada global, distribuyendo el tráfico hacia las instancias disponibles del Web Banking y del API Gateway. Además de optimizar la latencia

mediante el enrutamiento inteligente, este componente proporciona protección mediante Web Application Firewall (WAF) y permite redirigir automáticamente el tráfico hacia recursos saludables cuando se detecta una indisponibilidad.

En la capa de aplicaciones, los microservicios se ejecutan sobre Azure Kubernetes Service (AKS), donde el tráfico es distribuido entre múltiples réplicas de cada servicio mediante los mecanismos de balanceo internos del clúster. Esto evita que una única instancia concentre toda la carga de trabajo y permite atender incrementos en la demanda sin degradar el servicio.

Para mantener la disponibilidad de la plataforma, AKS implementa capacidades de auto-healing, supervisando continuamente el estado de los contenedores mediante Liveness Probes, Readiness Probes y Startup Probes. Cuando una instancia presenta fallos, deja de responder o no cumple las condiciones de salud definidas, Kubernetes la retira automáticamente del balanceador de carga y crea una nueva instancia para restablecer el servicio, sin intervención manual.

Adicionalmente, la solución incorpora Horizontal Pod Autoscaler (HPA) para incrementar o reducir automáticamente el número de réplicas de los microservicios en función de métricas como utilización de CPU, memoria o indicadores de negocio expuestos mediante Prometheus. De esta manera, la plataforma puede adaptarse dinámicamente a variaciones en la demanda, optimizando tanto el rendimiento como el consumo de recursos.

Estas capacidades se complementan con patrones de resiliencia implementados en los servicios de negocio, como Circuit Breaker, Retry, Timeout e Idempotencia, permitiendo aislar fallos en sistemas externos (Core Bancario, BANRED, BCE o proveedores de notificaciones) y evitando que una falla puntual comprometa la disponibilidad de toda la solución.

Como resultado, la arquitectura proporciona una plataforma altamente disponible, capaz de recuperarse automáticamente ante fallos de infraestructura o de aplicaciones, mantener la continuidad del servicio durante picos de carga y reducir significativamente los tiempos de recuperación (MTTR), contribuyendo al cumplimiento de los objetivos de disponibilidad y continuidad operativa definidos para la banca digital.

5.3. Red y seguridad perimetral

Azure Front Door se sitúa delante de Azure API Management como punto de entrada global, aportando protección contra ataques distribuidos de denegación de servicio y terminación TLS en el borde de la red antes de que el tráfico alcance la región donde se despliega la solución. Dentro de la región, AKS, Azure Cache for Redis y Azure SQL Database se despliegan en una red virtual con subredes segregadas por capa, de forma que el tráfico entre Middleware y la capa de datos no transite por una red pública, y las reglas de grupos de seguridad de red limitan explícitamente qué subred puede iniciar conexión hacia cuál.

5.4. Optimización de costos en la nube

Dado que la entidad cuenta con presupuesto de nube orientado a garantizar baja latencia, la arquitectura tecnológica debe traducir ese presupuesto en un uso eficiente de los servicios administrados de Azure, evitando tanto el sobre aprovisionamiento como decisiones de ahorro que comprometan la disponibilidad exigida por el ejercicio.

5.4.1. *Servicios administrados frente a autogestionados*

La arquitectura privilegia servicios PaaS de Azure —Azure Cache for Redis frente a un clúster Redis autogestionado, Azure SQL Database frente a un motor de base de datos sobre máquinas virtuales, y Azure Managed Grafana junto con Azure Monitor managed service for Prometheus frente a operar ese stack de observabilidad por cuenta propia—. Con ello, el parcheo, las actualizaciones, la alta disponibilidad interna y el respaldo de cada componente pasan a ser responsabilidad del proveedor. El precio por hora de un servicio administrado suele ser superior al de la infraestructura equivalente sin gestionar, pero el costo total de propiedad es menor una vez que se contabiliza el esfuerzo de un equipo de plataforma dedicado a sostener esos componentes por sí mismo; en esta arquitectura ese esfuerzo se redirige a construir capacidades de negocio en lugar de a operar herramientas de base.

5.4.2. *Modelo de consumo: elástico, serverless y capacidad reservada*

El cómputo de la plataforma combina tres modelos de consumo según el perfil de cada carga. Los servicios de Middleware se ejecutan sobre Azure Kubernetes Service con autoescalado horizontal, de modo que la capacidad crece durante los picos de demanda -pago de nómina, cierre de mes- y se contrae en horarios de baja actividad, evitando pagar de forma permanente por la capacidad que solo se requiere en las horas de mayor tráfico. El Procesamiento Batch y los adaptadores hacia BANRED y hacia el proveedor de notificaciones se llevaron deliberadamente a Azure Functions: son cargas de ejecución breve y frecuencia acotada, y un

modelo de pago por ejecución evita mantener cómputo activo la mayor parte del día para componentes que solo procesan en ventanas puntuales. Esta decisión se limita, sin embargo, a procesos donde la latencia de arranque en frío es tolerable; los servicios sensibles a la latencia del ejercicio -Autenticación, Cuentas, Transferencias- permanecen en AKS con instancias siempre activas para no introducir demoras en la ruta crítica del cliente. Finalmente, sobre la fracción de capacidad que se mantiene encendida de forma continua -el mínimo de nodos de AKS, Azure SQL Database y Azure Cache for Redis- se recomienda contratar Reservas de Azure o Planes de Ahorro, que reducen de forma significativa el precio frente al pago por uso a cambio de un compromiso de consumo, dejando el pago por uso para la capacidad variable y para las cargas serverless..

5.4.3. Gobierno y visibilidad de costos (FinOps)

Se recomienda etiquetar todo recurso desplegado en Azure por dominio de negocio -Consultas, Transferencias, Autenticación, Onboarding, Observabilidad- y explotar esas etiquetas en Azure Cost Management, de forma que el costo de la plataforma pueda desagregarse por capacidad de negocio y no leerse únicamente como una cifra total de infraestructura.

Sobre esa base se configuran presupuestos y alertas de consumo por dominio, para que un crecimiento inesperado en un componente puntual -por ejemplo, un aumento del volumen de logs enviado a observabilidad- se detecte antes de convertirse en una desviación presupuestaria significativa. La observabilidad recibe además una política de retención escalonada: los logs y trazas de los últimos días permanecen en almacenamiento de alto desempeño para el diagnóstico de incidentes recientes, mientras que la retención de largo plazo, orientada a auditoría, se traslada a niveles de almacenamiento de menor costo, suficientes para esa necesidad pero sin sostener indefinidamente el precio del almacenamiento de alto desempeño. Como práctica continua, se recomienda revisar periódicamente el dimensionamiento (right-sizing) de los recursos frente a su utilización real, ajustando los SKU sobredimensionados que el propio etiquetado y los reportes de costo permitan identificar..

6. Patrones Arquitectónicos

La selección de patrones de esta arquitectura no responde a una lista de buenas prácticas genéricas sino a los atributos de calidad que el ejercicio exige de manera explícita: alta disponibilidad, tolerancia a fallos y desempeño de las consultas. La recuperación ante desastres, por su naturaleza, no se resuelve mediante un patrón de diseño de aplicación sino en la capa de infraestructura: redundancia multi-región en modelo warm standby, descrita en la Arquitectura Tecnológica, por lo que

queda fuera de este catálogo. Cada patrón descrito a continuación se seleccionó para resolver un problema puntual identificado en los capítulos anteriores.

6.1. API Gateway

Actúa como fachada única entre los canales digitales y los servicios de negocio, centralizando la autenticación, el enrutamiento y el control de tráfico. Se seleccionó porque el ejercicio exige explícitamente que la capa de integración esté compuesta por un API Gateway, y porque sin este patrón cada canal debería conocer la ubicación y el contrato de cada servicio individual, lo que impediría evolucionar los servicios de Middleware sin afectar a los canales.

6.2. Circuit Breaker

Se aplica sobre las integraciones hacia BANRED, hacia el BCE y hacia los proveedores de notificación, todos ellos actores externos sobre los que banco no tiene control operativo. Cuando alguno de estos actores comienza a fallar o a responder con latencias anormales, el circuito se abre y la solución deja de intentar la llamada por un período determinado, devolviendo una respuesta controlada al canal en lugar de dejar que la falla externa se propague y agote los recursos de los servicios de Middleware.

6.3. Retry

Se complementa con el Circuit Breaker para las operaciones idempotentes -principalmente consultas- en las que un fallo transitorio de red puede resolverse con un segundo intento antes de considerar la operación como fallida. Se recomienda limitar el número de reintentos y aplicar una espera incremental entre ellos (backoff exponencial), de forma que el patrón no agrave una situación de saturación del sistema externo que ya está fallando. Sobre el camino de escritura -en particular la ejecución de transferencias-, el reintento solo es seguro cuando se combina con el patrón de Idempotencia descrito más adelante, que garantiza que un segundo intento no materialice dos veces la misma operación.

6.4. Bulkhead

Se recomienda aislar los recursos -conexiones, hilos de ejecución- que cada integración externa consume, de forma que una degradación en la integración con BANRED no agote los recursos disponibles para la integración con el BCE o con los proveedores de notificación. Sin este aislamiento, el Circuit Breaker por sí solo no evita que una integración lenta consuma la totalidad de la capacidad del sistema antes de que el circuito llegue a abrirse.

6.5. Cache aside

Resuelve el mecanismo de persistencia para clientes frecuentes solicitado en el ejercicio: el servicio consulta primero la caché, y solo ante una ausencia o

expiración del dato acude al sistema de registro o a Stratio, actualizando la caché para las siguientes solicitudes. Este patrón reduce la carga sobre el Core Bancario y mejora la latencia percibida por el canal en las consultas más frecuentes, como el resumen financiero y el histórico de movimientos. Conviene precisar que este patrón opera en un nivel distinto al de CQRS descrito más adelante: CQRS separa los modelos y caminos de lectura y escritura a nivel de diseño, mientras que Cache Aside es el mecanismo concreto que acelera ese camino de lectura ya separado; ambos son complementarios y no redundantes.

6.6. CQRS

Se recomienda separar el camino de lectura del camino de escritura para el servicio de cuentas y movimientos: las consultas se resuelven contra Stratio y la capa de caché, mientras que las operaciones que modifican el estado como por ejemplo: una transferencia, se dirigen siempre y exclusivamente al Core Bancario. Esta separación es coherente con la decisión de arquitectura de datos descrita anteriormente y evita que un mismo modelo de datos deba optimizarse simultáneamente para consulta masiva y para consistencia transaccional estricta.

6.7. Backend for Frontend (BFF)

Se recomienda anteponer a los canales una capa intermedia específica por tipo de cliente, de forma que el Web Banking y la App Móvil no consuman directamente los servicios de negocio sino un backend adaptado a cada uno. Este patrón se seleccionó porque el ejercicio exige dos canales con necesidades diferenciadas — una SPA y una aplicación móvil multiplataforma— y cada uno impone requisitos distintos sobre la forma de los datos y, sobre todo, sobre el manejo de las credenciales: para el Web Banking, el BFF retiene los tokens OAuth del lado del servidor y expone al navegador únicamente una cookie de sesión httpOnly, reduciendo la superficie de robo de tokens ante un eventual XSS, tal como se describe en la Arquitectura de Autenticación. Al concentrar en el BFF la adaptación de respuestas y la orquestación propia de cada canal, se evita sobrecargar el API Gateway con lógica específica de presentación y se permite que cada canal evolucione su experiencia sin forzar cambios en los servicios de negocio compartidos.

6.8. Idempotencia (clave de idempotencia)

Se aplica sobre las operaciones que modifican estado, de manera prioritaria sobre la ejecución de transferencias, para garantizar que una misma operación no se materialice dos veces aunque la solicitud llegue repetida. Este patrón es la condición que hace seguro el uso de Retry y de Circuit Breaker sobre el camino de escritura: sin él, un reintento tras un fallo transitorio de red podría duplicar un débito, ya que el canal no tiene forma de saber si la operación original llegó a

completarse antes de la interrupción. El mecanismo consiste en que el canal genera un identificador único por operación —la clave de idempotencia— que acompaña la solicitud; el servicio de transferencias registra esa clave junto con el resultado de la operación, y ante la llegada de una solicitud con una clave ya procesada, devuelve el resultado previamente calculado en lugar de ejecutar nuevamente la transferencia. De esta forma la solución tolera reintentos del canal y de los mecanismos de resiliencia sin comprometer la integridad financiera de la operación.

6.9. Event Driven

Se recomienda para el flujo de notificaciones y para la actualización de la caché de consulta, en lugar de que el servicio de transferencias invoque de forma síncrona al servicio de notificaciones, la confirmación de una transferencia debe publicar un evento que el servicio de notificaciones consume de forma asíncrona, y ese mismo evento puede utilizarse para invalidar o refrescar la entrada correspondiente en la caché de movimientos. Este patrón desacopla la ejecución de la transferencia -que debe ser rápida y confiable- de las acciones secundarias que derivan de ella.

6.10. Transactional Outbox

Se recomienda para garantizar que la publicación de eventos ocurra de forma atómica con el cambio de estado que los origina, resolviendo el problema de la doble escritura implícito en el patrón Event Driven. Cuando un servicio debe simultáneamente persistir un cambio en su base de datos y publicar un evento hacia el bus de mensajería, hacerlo en dos operaciones independientes abre una ventana de inconsistencia: si el servicio confirma la transferencia pero falla antes de publicar el evento, el cliente no es notificado y la caché de movimientos queda obsoleta; si publica el evento pero la transacción no se confirma, se comunica una operación que nunca ocurrió. El patrón elimina esa ventana escribiendo el evento en una tabla outbox dentro de la misma transacción de base de datos que el cambio de estado; un proceso publicador independiente —o un mecanismo de captura de cambios— lee esa tabla y publica los eventos hacia el bus con garantía de entrega al menos una vez, apoyándose en la idempotencia de los consumidores para tolerar reenvíos.

Este patrón cumple además una segunda función dentro de la solución: es el mecanismo que relaciona los componentes del registro de auditoría solicitado por el ejercicio. Cada acción relevante del cliente —inicio de sesión, transferencia, cambio de credenciales— se escribe en la tabla outbox del servicio que la ejecuta dentro de su propia transacción de negocio, y el publicador la traslada de forma asíncrona hacia la base de datos de auditoría. Así se garantiza que ninguna operación efectivamente ejecutada quede sin su registro de auditoría, sin que el servicio de negocio dependa de una escritura síncrona contra la auditoría para

completar su operación, preservando el desacoplamiento descrito en la Arquitectura de Datos.

Justificación de las Decisiones Arquitectónicas

Cada uno de los argumentos que sustentaron esta propuesta responde a una tensión concreta entre lo que el negocio necesita y lo que los sistemas existentes pueden ofrecer, y es esa tensión, más que una preferencia técnica, la que explica por qué la arquitectura se construyó de la manera en que se construyó.

La decisión de partir de una Arquitectura Empresarial, y no de un diseño de aplicaciones directo, respondió a la necesidad de que el requerimiento de banca digital se planteara primero como parte del negocio del banco y no como un proyecto de tecnología aislado; por esa razón el diagrama de contexto de este ejercicio no se construyó dibujando cajas de sistemas sino derivándolo de los flujos de valor y del mapa de capacidades ya validados, lo que garantiza que cada servicio expuesto en el contexto tenga un dueño de negocio identificable y una razón de existir más allá de la conveniencia técnica.

La implementación del patrón Circuit Breaker en los servicios respondió a que la solución depende de actores externos -BANRED, BCE, proveedores de notificación- sobre los cuales BP no tiene ni visibilidad ni control operativo; sin este patrón, una falla de cualquiera de esos actores se propagaría hacia los servicios de Middleware y, de ahí, hacia la experiencia completa del canal, incluso para clientes que no estuvieran intentando una operación relacionada con el actor caído.

La incorporación de Stratio como fuente de consulta para el Core Bancario respondió a que este último es, por su propia naturaleza, un sistema de alta demanda que atiende simultáneamente la operación presencial, la operación de otros canales de la entidad y ahora también la operación del canal digital; sin una capa intermedia que absorba el volumen de consultas de los diferentes canales, cualquier pico de uso del canal digital habría competido directamente por los mismos recursos que sostienen la operación transaccional crítica del banco.

Conclusiones

La arquitectura presentada satisface el requerimiento central del ejercicio -una plataforma de banca por internet que permita consultar productos y ejecutar

transferencias locales e interbancarias- porque cada decisión técnica descrita en este documento puede rastrearse hasta un flujo de valor, una capacidad de negocio o un driver regulatorio identificado en el modelo de Arquitectura Empresarial, y no hasta una preferencia tecnológica aislada del arquitecto.

Los requerimientos no funcionales de alta disponibilidad, tolerancia a fallos y recuperación ante desastres quedan resueltos mediante la combinación de redundancia regional y zonal en la infraestructura de nube, el patrón Circuit Breaker en las integraciones externas, y la orquestación de contenedores con capacidad de auto-healing; la exigencia de baja latencia en las consultas se resuelve mediante Stratio y el patrón Cache Aside, que evitan que el canal digital compita directamente con la operación transaccional del Core Bancario.